

Imports

```
In [399... import calendar
from datetime import datetime

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import missingno as msno
import seaborn as sns
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
from xgboost import XGBRegressor
import optuna
from optuna.samplers import TPESampler
```

Bike Sharing Demand Prediction

1. Load Data

Loading both training and test datasets from the Kaggle Bike Sharing Demand competition.

```
In [400... train_df = pd.read_csv("train.csv")
test_df = pd.read_csv("test.csv")
```

2. Data Summary

- Dataset Shape and First Rows
- Data Types and Info
- Descriptive Statistics
- Missing Values

```
In [401... print(f"Train shape: {train_df.shape}")
print(f"Test shape: {test_df.shape}")
train_df.head()
```

```
Train shape: (10886, 12)
Test shape: (6493, 9)
```

Out [401...

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	wind:
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	
3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395	75	
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395	75	

In [402... `train_df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   datetime        10886 non-null  object
1   season          10886 non-null  int64
2   holiday         10886 non-null  int64
3   workingday      10886 non-null  int64
4   weather         10886 non-null  int64
5   temp           10886 non-null  float64
6   atemp           10886 non-null  float64
7   humidity        10886 non-null  int64
8   windspeed       10886 non-null  float64
9   casual          10886 non-null  int64
10  registered      10886 non-null  int64
11  count           10886 non-null  int64
dtypes: float64(3), int64(8), object(1)
memory usage: 1020.7+ KB
```

In [403... `train_df.describe()`

Out [403...

	season	holiday	workingday	weather	temp	
count	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000
mean	2.506614	0.028569	0.680875	1.418427	20.23086	
std	1.116174	0.166599	0.466159	0.633839	7.79159	
min	1.000000	0.000000	0.000000	1.000000	0.82000	
25%	2.000000	0.000000	0.000000	1.000000	13.94000	
50%	3.000000	0.000000	1.000000	1.000000	20.50000	
75%	4.000000	0.000000	1.000000	2.000000	26.24000	
max	4.000000	1.000000	1.000000	4.000000	41.00000	

3. Data Transformation

As we see from the above results, the columns "season", "holiday", "workingday" and "weather" should be of "categorical" data type. But the current data type is "int" for those columns.

Let us transform the dataset in the following ways:

- Create new columns "date", "hour", "weekDay", "month" from "datetime" column
- Coerce the datatype of "season", "holiday", "workingday" and "weather" to category
- Drop the datetime column as we already extracted useful features from it

In [404...

```
season_map = {1: "Spring", 2: "Summer", 3: "Fall", 4: "Winter"}

weather_map = {
    1: "Clear + Few clouds + Partly cloudy",
    2: "Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist",
    3: "Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light R",
    4: "Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog"
}

def extract_date(dt_string):
    return dt_string.split()[0]

def extract_hour(dt_string):
    return dt_string.split()[1].split(":")[0]

def extract_year(dt_string):
    return dt_string.split()[0].split("-")[0]

def get_weekday_name(date_string):
    return calendar.day_name[datetime.strptime(date_string, "%Y-%m-%d").w

def get_month_name(date_string):
    return calendar.month_name[datetime.strptime(date_string, "%Y-%m-%d").m

def transform_features(df):
```

```

"""Extract and transform features from datetime column."""
df = df.copy()
df["date"] = df["datetime"].apply(extract_date)
df["hour"] = df["datetime"].apply(extract_hour)
df["year"] = df["datetime"].apply(extract_year)
df["weekday"] = df["date"].apply(get_weekday_name)
df["month"] = df["date"].apply(get_month_name)
df["season"] = df["season"].map(season_map)
df["weather"] = df["weather"].map(weather_map)
return df.drop(columns=["datetime"])

def convert_to_category(df, columns):
    """Convert specified columns to category dtype."""
    df = df.copy()
    for col in columns:
        df[col] = df[col].astype("category")
    return df

categorical_cols = ["hour", "weekday", "month", "season", "weather", "hol

train_df = (train_df
            .pipe(transform_features)
            .pipe(convert_to_category, categorical_cols))

test_df = (test_df
          .pipe(transform_features)
          .pipe(convert_to_category, categorical_cols))

```

In [405... train_df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 16 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   season          10886 non-null  category
 1   holiday         10886 non-null  category
 2   workingday      10886 non-null  category
 3   weather         10886 non-null  category
 4   temp            10886 non-null  float64
 5   atemp           10886 non-null  float64
 6   humidity        10886 non-null  int64
 7   windspeed       10886 non-null  float64
 8   casual          10886 non-null  int64
 9   registered      10886 non-null  int64
10   count           10886 non-null  int64
11   date            10886 non-null  object
12   hour            10886 non-null  category
13   year            10886 non-null  category
14   weekday         10886 non-null  category
15   month           10886 non-null  category
dtypes: category(8), float64(3), int64(4), object(1)
memory usage: 767.8+ KB

```

In [406... train_df.head()

Out [406...

	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	cas
0	Spring	0	0	Clear + Few clouds + Partly cloudy	9.84	14.395	81	0.0	
1	Spring	0	0	Clear + Few clouds + Partly cloudy	9.02	13.635	80	0.0	
2	Spring	0	0	Clear + Few clouds + Partly cloudy	9.02	13.635	80	0.0	
3	Spring	0	0	Clear + Few clouds + Partly cloudy	9.84	14.395	75	0.0	
4	Spring	0	0	Clear + Few clouds + Partly cloudy	9.84	14.395	75	0.0	

4. Exploratory Data Analysis

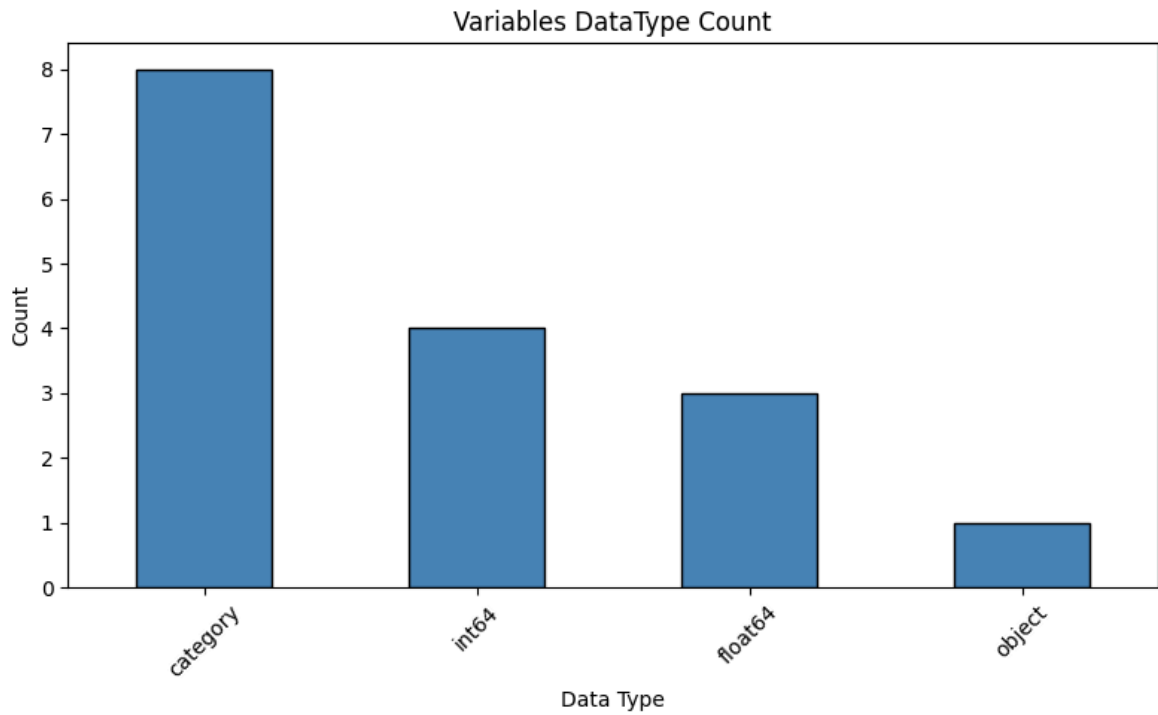
4.1 Variables DataType Count

Let's start with a simple visualization of the count of variables by their data type.

In [407...

```
dtype_counts = train_df.dtypes.astype(str).value_counts()

fig, ax = plt.subplots(figsize=(8, 5))
dtype_counts.plot(kind="bar", ax=ax, color="steelblue", edgecolor="black")
ax.set_xlabel("Data Type")
ax.set_ylabel("Count")
ax.set_title("Variables DataType Count")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

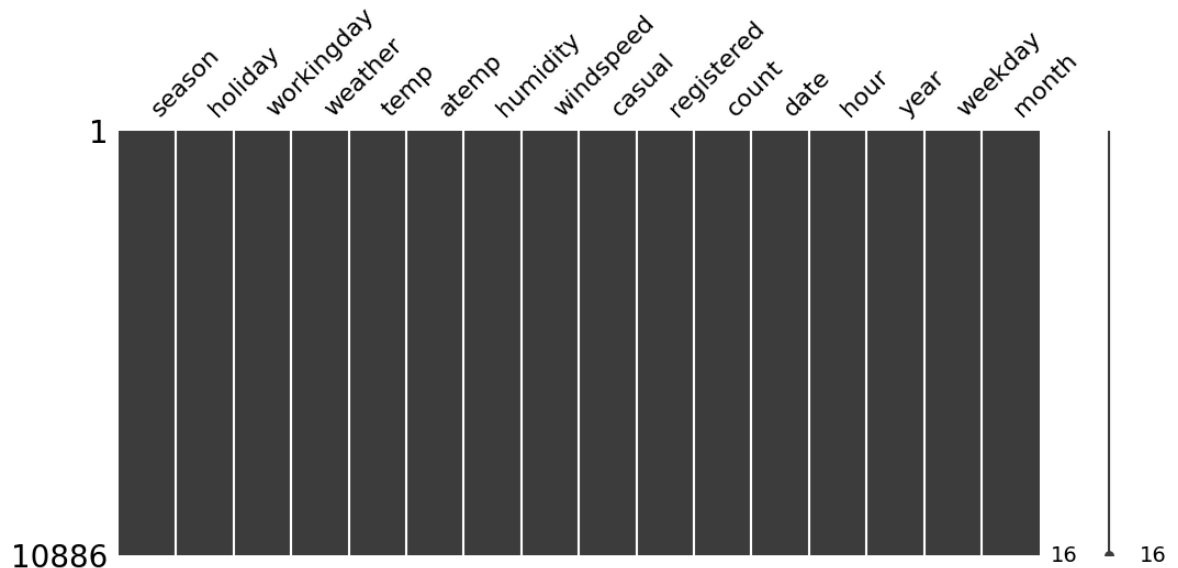


4.2 Missing Values Visualization

Using missingno to visualize the distribution of missing values.

```
In [408...] msno.matrix(train_df, figsize=(12, 5))
```

```
Out [408...] <Axes: >
```



4.3 Outliers Analysis

At first look, "count" variable contains lot of outlier data points which skews the distribution towards right (as there are more data points beyond Outer Quartile Limit). Following inferences can be made from the boxplots below:

- Spring season has got relatively lower count. The dip in median value in boxplot gives evidence for it.

- The boxplot with "Hour Of The Day" is quite interesting. The median values are relatively higher at 7AM - 8AM and 5PM - 6PM. It can be attributed to regular school and office users at that time.
- Most of the outlier points are mainly contributed from "Working Day" than "Non Working Day".

```
In [409... fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(14, 10))

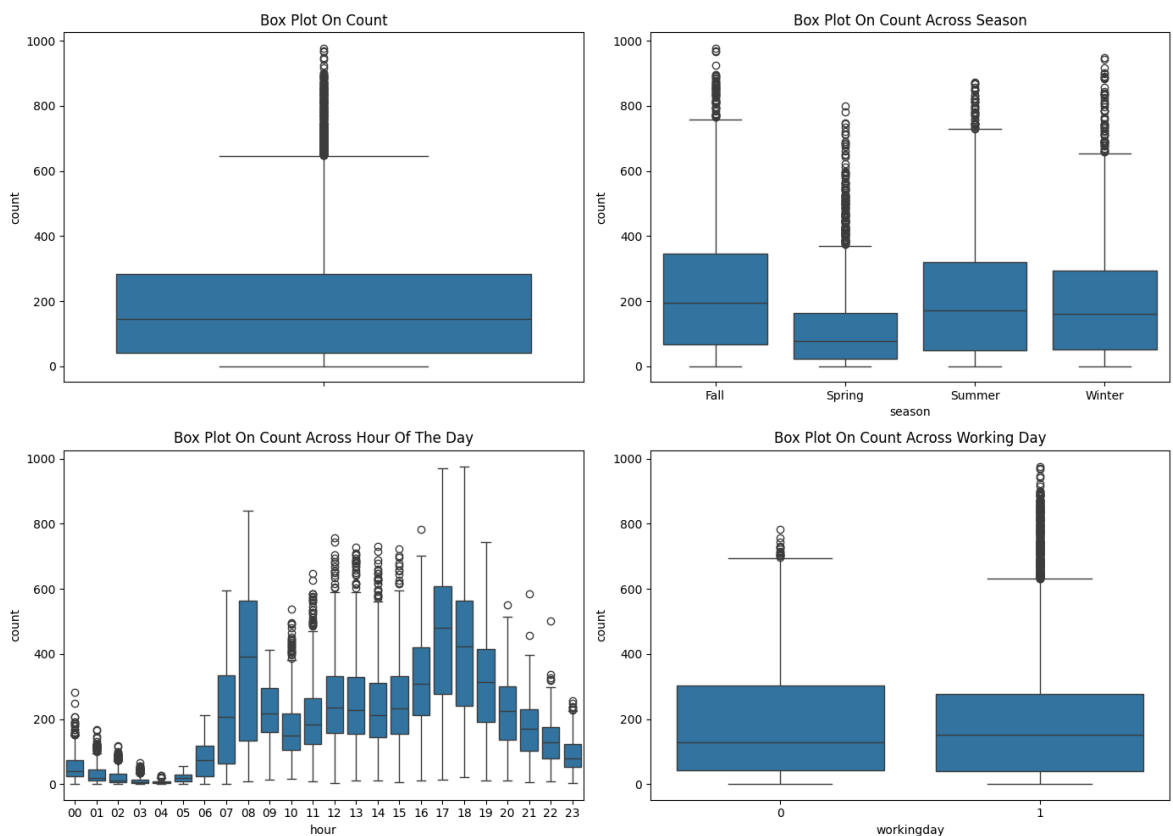
sns.boxplot(data=train_df, y="count", ax=axes[0, 0])
axes[0, 0].set_title("Box Plot On Count")

sns.boxplot(data=train_df, x="season", y="count", ax=axes[0, 1])
axes[0, 1].set_title("Box Plot On Count Across Season")

sns.boxplot(data=train_df, x="hour", y="count", ax=axes[1, 0])
axes[1, 0].set_title("Box Plot On Count Across Hour Of The Day")

sns.boxplot(data=train_df, x="workingday", y="count", ax=axes[1, 1])
axes[1, 1].set_title("Box Plot On Count Across Working Day")

plt.tight_layout()
plt.show()
```



4.4 Correlation Analysis

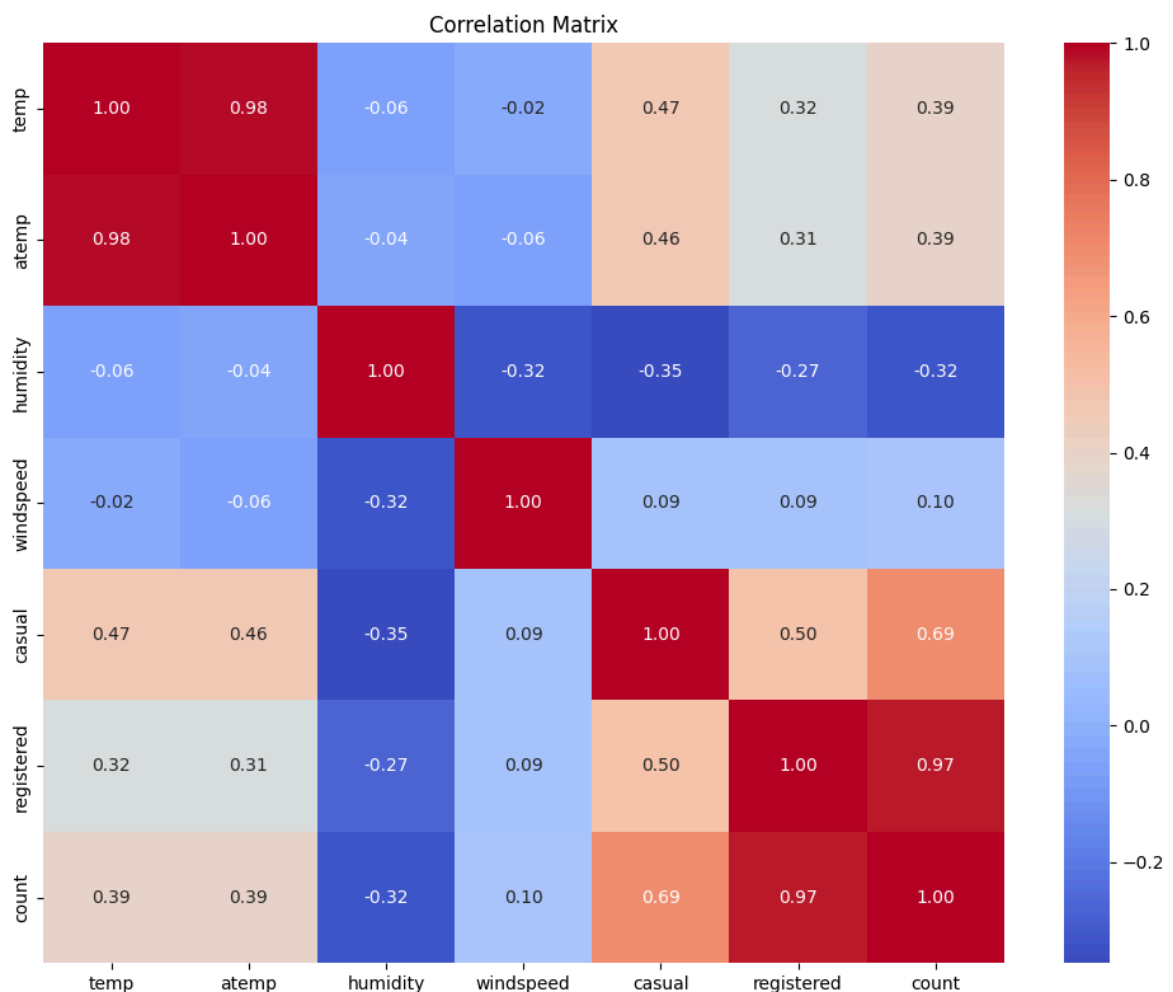
One common way to understand how a dependent variable is influenced by features (numerical) is to find a correlation matrix between them.

- temp, atemp and humidity features have positive and negative correlation with count respectively.

- windspeed is not gonna be really useful numerical feature and it is visible from its correlation value with "count".
- Since "atemp" and "temp" have strong correlation with each other, meaning one of them should be dropped during model building
- "Casual" and "Registered" are not taken into account since they are leakage variables in nature and need to be dropped during model building.

```
In [410]: corr_matrix = train_df.select_dtypes(include=["int64", "float64"]).corr()

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", ax=ax)
ax.set_title("Correlation Matrix")
plt.tight_layout()
plt.show()
```



Regression plot in seaborn is one useful way to depict the relationship between two features. Here we consider "count" vs "temp", "humidity", "windspeed".

```
In [411]: fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(15, 5))

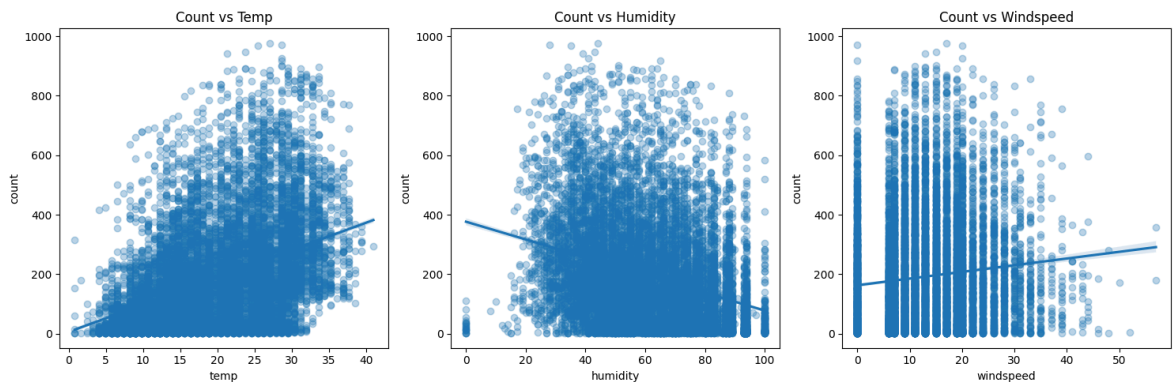
sns.regplot(data=train_df, x="temp", y="count", ax=axes[0], scatter_kws={
    axes[0].set_title("Count vs Temp")

sns.regplot(data=train_df, x="humidity", y="count", ax=axes[1], scatter_k
axes[1].set_title("Count vs Humidity")

sns.regplot(data=train_df, x="windspeed", y="count", ax=axes[2], scatter_
```

```
axes[2].set_title("Count vs Windspeed")
```

```
plt.tight_layout()
plt.show()
```



4.5 Visualizing Distribution Of Data

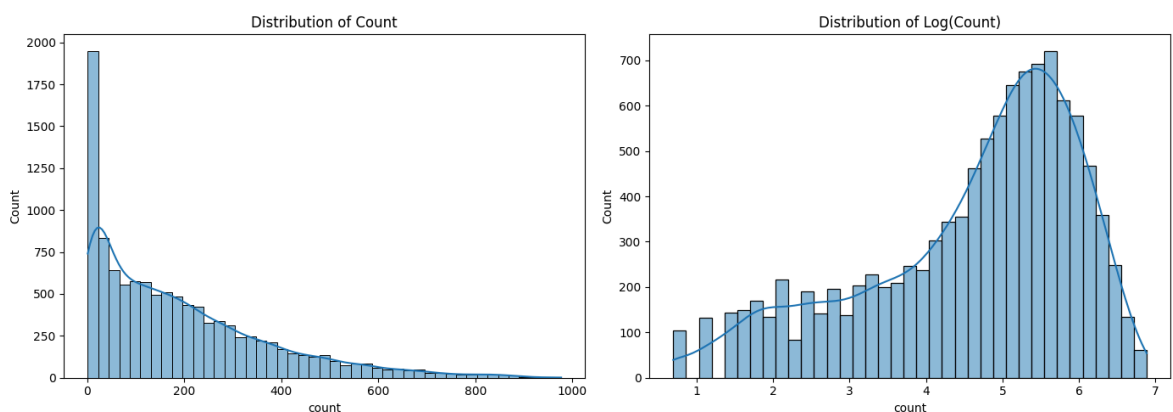
As it is visible from the below figures that "count" variable is skewed towards right. It is desirable to have Normal distribution as most of the machine learning techniques require dependent variable to be Normal. One possible solution is to take log transformation on "count" variable. After the transformation the data looks lot better but still not ideally following normal distribution.

```
In [412... fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(14, 5))

sns.histplot(train_df["count"], kde=True, ax=axes[0])
axes[0].set_title("Distribution of Count")

sns.histplot(np.log1p(train_df["count"]), kde=True, ax=axes[1])
axes[1].set_title("Distribution of Log(Count)")

plt.tight_layout()
plt.show()
```



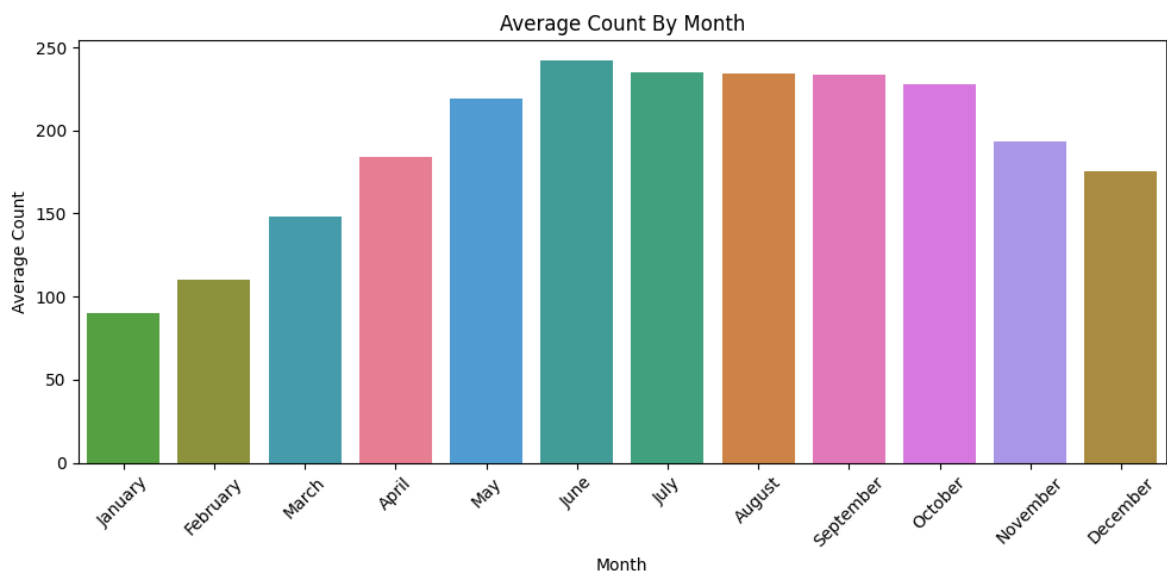
4.6 Visualizing Count Vs (Month, Season, Hour, Weekday, Usertype)

- It is quite obvious that people tend to rent bike during summer season since it is really conducive to ride bike at that season. Therefore June, July and August have got relatively higher demand for bicycle.

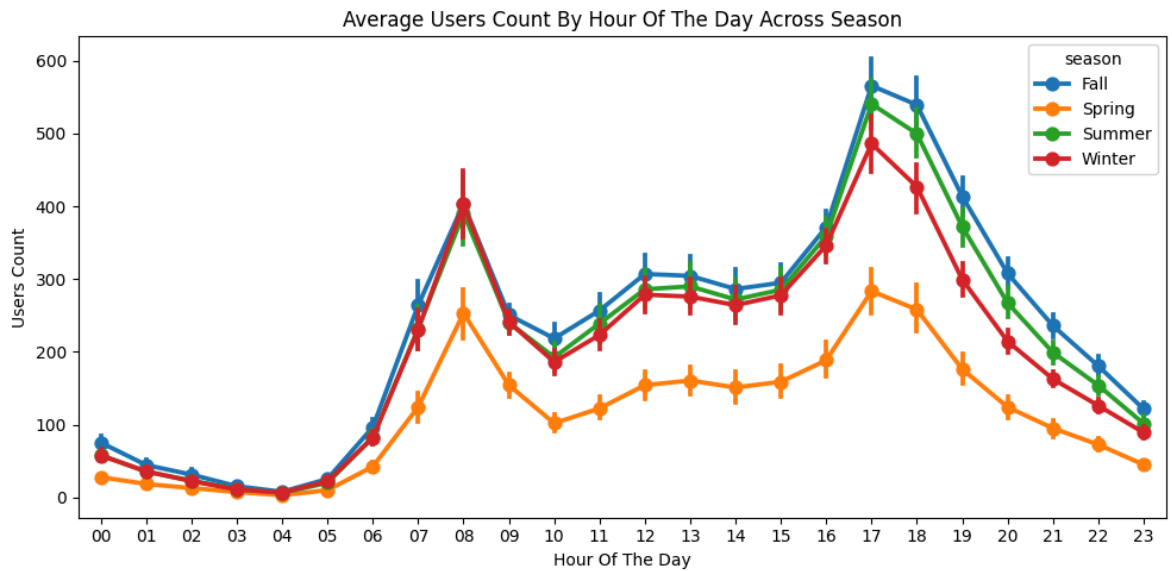
- On weekdays more people tend to rent bicycle around 7AM-8AM and 5PM-6PM. As we mentioned earlier this can be attributed to regular school and office commuters.
- Above pattern is not observed on "Saturday" and "Sunday". More people tend to rent bicycle between 10AM and 4PM.
- The peak user count around 7AM-8AM and 5PM-6PM is purely contributed by registered users.

```
In [413... month_order = ["January", "February", "March", "April", "May", "June",
                "July", "August", "September", "October", "November", "Dec

fig, ax = plt.subplots(figsize=(10, 5))
sns.barplot(data=train_df, x="month", y="count", ax=ax, errorbar=None, or
            hue="month", palette="husl", legend=False)
ax.set_title("Average Count By Month")
ax.set_xlabel("Month")
ax.set_ylabel("Average Count")
ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```

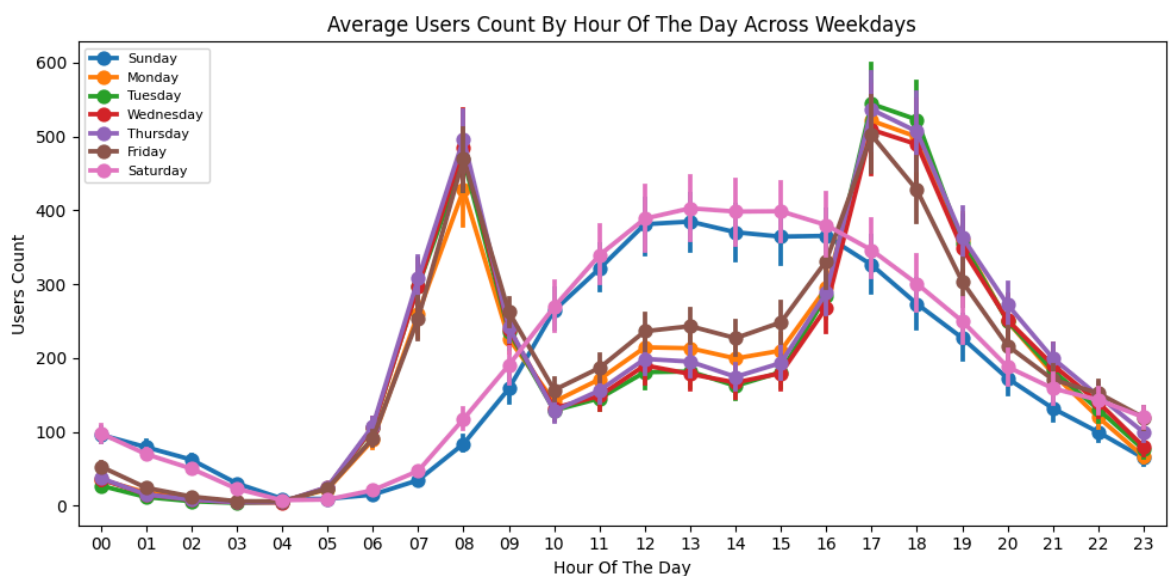


```
In [414... fig, ax = plt.subplots(figsize=(10, 5))
sns.pointplot(data=train_df, x="hour", y="count", hue="season", ax=ax)
ax.set_title("Average Users Count By Hour Of The Day Across Season")
ax.set_xlabel("Hour Of The Day")
ax.set_ylabel("Users Count")
plt.tight_layout()
plt.show()
```



```
In [415... weekday_order = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday",

fig, ax = plt.subplots(figsize=(10, 5))
sns.pointplot(data=train_df, x="hour", y="count", hue="weekday", hue_order=
ax.set_title("Average Users Count By Hour Of The Day Across Weekdays")
ax.set_xlabel("Hour Of The Day")
ax.set_ylabel("Users Count")
ax.legend(loc="upper left", fontsize=8)
plt.tight_layout()
plt.show()
```



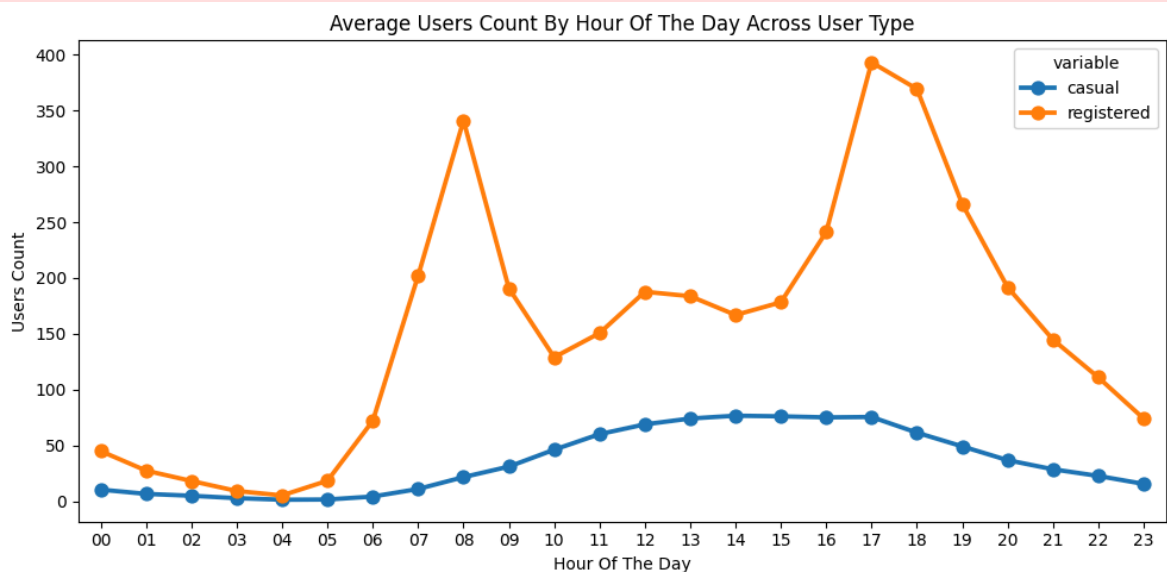
```
In [416... hourly_data = train_df.groupby("hour")[["casual", "registered"]].mean().r
hourly_melted = hourly_data.melt(id_vars="hour", var_name="variable", val

fig, ax = plt.subplots(figsize=(10, 5))
sns.pointplot(data=hourly_melted, x="hour", y="count", hue="variable", ax
ax.set_title("Average Users Count By Hour Of The Day Across User Type")
ax.set_xlabel("Hour Of The Day")
ax.set_ylabel("Users Count")
plt.tight_layout()
plt.show()
```

```

/var/folders/mp/ttyv6tg92c74prtz539th3qc0000gn/T/ipykernel_19586/67810193
0.py:1: FutureWarning: The default of observed=False is deprecated and will
be changed to True in a future version of pandas. Pass observed=False to
retain current behavior or observed=True to adopt the future default and s
ilence this warning.
hourly_data = train_df.groupby("hour")[["casual", "registered"]].mean().
reset_index()

```



5. Model Building

5.1 Data Preparation

Before building models, we need to:

- Save test datetime for submission
- Drop leakage columns (`casual` , `registered`) - they sum to `count` and are not available in test data
- Drop `date` column - we already extracted useful features from it
- Log-transform the target variable (`count`) to handle right-skewness
- Convert categorical variables to dummy variables for linear models

```

In [417]: test_datetime = pd.read_csv("test.csv")["datetime"]

# Drop leakage columns, date, and atemp (highly correlated with temp)
drop_columns = ["casual", "registered", "date", "count", "atemp"]

y_train = np.log1p(train_df["count"])

X_train = train_df.drop(columns=drop_columns)
X_test = test_df.drop(columns=["date", "atemp"])

X_train = pd.get_dummies(X_train, drop_first=True)
X_test = pd.get_dummies(X_test, drop_first=True)

X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

print(f"X_train shape: {X_train.shape}")

```

```
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
```

```
X_train shape: (10886, 52)
X_test shape: (6493, 52)
y_train shape: (10886,)
```

5.2 RMSLE Metric

The Kaggle competition uses Root Mean Squared Logarithmic Error (RMSLE) as the evaluation metric.

```
In [418.. def rmsle(y_true, y_pred):
    """Calculate RMSLE between actual and predicted values."""
    return np.sqrt(mean_squared_error(y_true, y_pred))
```

5.3 Model Training and Evaluation

For each model, we will:

1. Train on the full training set
2. Calculate RMSLE on training data
3. Show actual vs predicted comparison on training data
4. Show distribution of predictions on test data

```
In [419.. from sklearn.model_selection import cross_val_score

def evaluate_model(model, model_name, X_train, y_train, X_test):
    """Train model, evaluate on train, and predict on test."""
    model.fit(X_train, y_train)

    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)

    train_rmsle = rmsle(y_train, y_pred_train)

    fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(18, 5))

    # Plot 1: Train actual vs predicted distribution
    sns.histplot(np.exp1(y_train), kde=True, ax=axes[0], label="Actual",
    sns.histplot(np.exp1(y_pred_train), kde=True, ax=axes[0], label="Pre
    axes[0].set_title(f"{model_name}: Train Distribution")
    axes[0].set_xlabel("Count")
    axes[0].legend()

    # Plot 2: Train actual vs predicted scatter
    axes[1].scatter(y_train, y_pred_train, alpha=0.3)
    axes[1].plot([y_train.min(), y_train.max()], [y_train.min(), y_train.
    axes[1].set_xlabel("Actual (log)")
    axes[1].set_ylabel("Predicted (log)")
    axes[1].set_title(f"{model_name}: Train Actual vs Predicted")

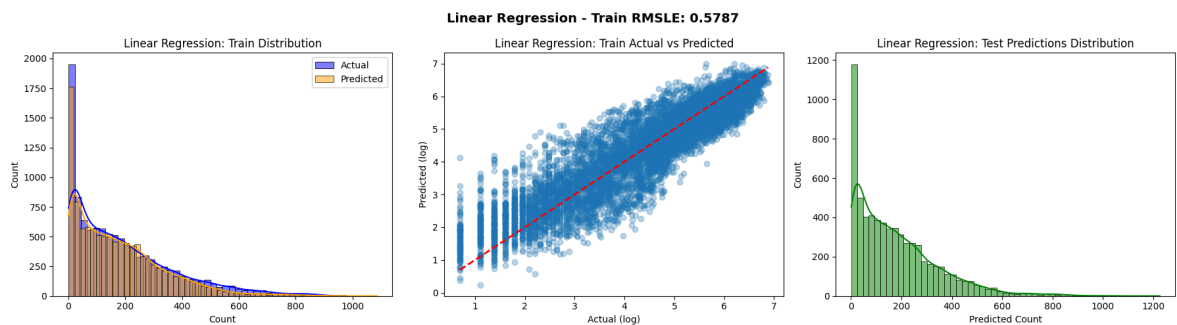
    # Plot 3: Test predictions distribution
    sns.histplot(np.exp1(y_pred_test), kde=True, ax=axes[2], color="gree
    axes[2].set_title(f"{model_name}: Test Predictions Distribution")
    axes[2].set_xlabel("Predicted Count")
```

```
plt.suptitle(f"{model_name} - Train RMSLE: {train_rmsle:.4f}", fontsize=14)
plt.tight_layout()
plt.show()

return y_pred_train, y_pred_test, train_rmsle
```

Linear Regression

```
In [420]: lr_model = LinearRegression()
y_pred_train_lr, y_pred_test_lr, rmsle_lr = evaluate_model(lr_model, "Lin
```



Ridge Regression

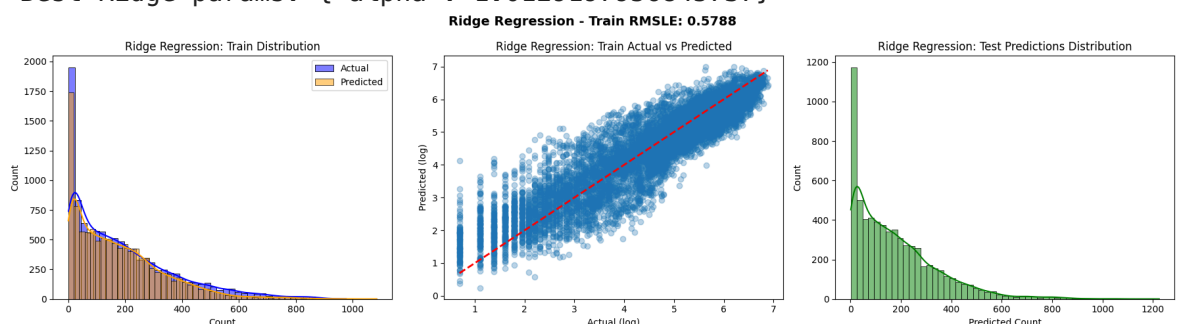
```
In [421]: def ridge_objective(trial):
alpha = trial.suggest_float("alpha", 0.001, 100, log=True)
model = Ridge(alpha=alpha)
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="neg_
return -scores.mean()

optuna.logging.set_verbosity(optuna.logging.WARNING)
ridge_study = optuna.create_study(direction="minimize", sampler=TPESampler)
ridge_study.optimize(ridge_objective, n_trials=30, show_progress_bar=True)

print(f"Best Ridge params: {ridge_study.best_params}")
ridge_model = Ridge(**ridge_study.best_params)
y_pred_train_ridge, y_pred_test_ridge, rmsle_ridge = evaluate_model(ridge
```

Best trial: 8. Best value: 0.612555: 100%|██████████| 30/30 [00:00<00:00, 53.62it/s]

Best Ridge params: {'alpha': 1.0129197956845737}



Lasso Regression

```
In [422]: def lasso_objective(trial):
alpha = trial.suggest_float("alpha", 0.0001, 1, log=True)
model = Lasso(alpha=alpha)
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="neg_
```

```

return -scores.mean()

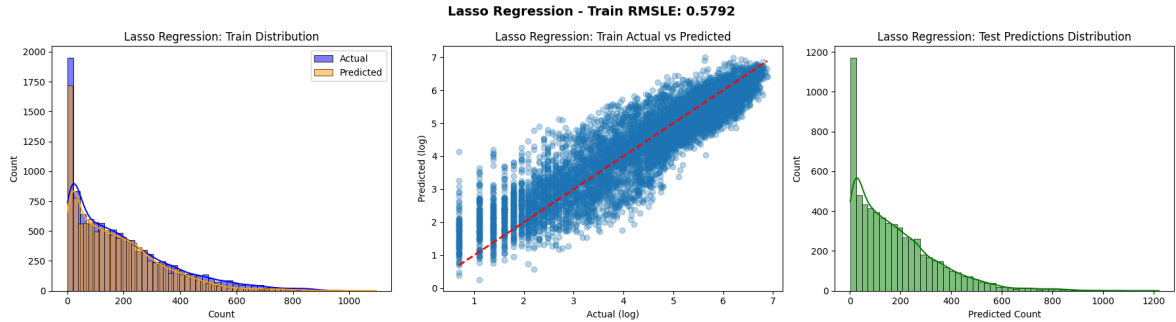
lasso_study = optuna.create_study(direction="minimize", sampler=TPESampler)
lasso_study.optimize(lasso_objective, n_trials=30, show_progress_bar=True)

print(f"Best Lasso params: {lasso_study.best_params}")
lasso_model = Lasso(**lasso_study.best_params)
y_pred_train_lasso, y_pred_test_lasso, rmsle_lasso = evaluate_model(lasso

```

Best trial: 21. Best value: 0.612113: 100%|██████████| 30/30 [00:03<00:00, 9.28it/s]

Best Lasso params: {'alpha': 0.00032446474808171187}



Random Forest

In [423...

```

def rf_objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 100, 500),
        "max_depth": trial.suggest_int("max_depth", 10, 50),
        "min_samples_split": trial.suggest_int("min_samples_split", 2, 10),
        "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 4),
        "random_state": 42,
        "n_jobs": -1
    }
    model = RandomForestRegressor(**params)
    scores = cross_val_score(model, X_train, y_train, cv=3, scoring="neg_
    return -scores.mean()

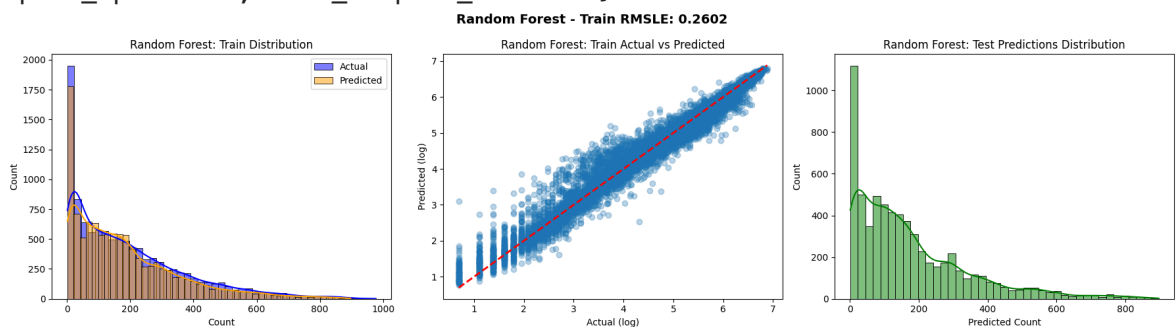
rf_study = optuna.create_study(direction="minimize", sampler=TPESampler(s
rf_study.optimize(rf_objective, n_trials=20, show_progress_bar=True)

print(f"Best Random Forest params: {rf_study.best_params}")
rf_model = RandomForestRegressor(**rf_study.best_params, random_state=42,
y_pred_train_rf, y_pred_test_rf, rmsle_rf = evaluate_model(rf_model, "Ran

```

Best trial: 18. Best value: 0.593252: 100%|██████████| 20/20 [00:31<00:00, 1.57s/it]

Best Random Forest params: {'n_estimators': 110, 'max_depth': 30, 'min_samples_split': 7, 'min_samples_leaf': 2}



XGBoost

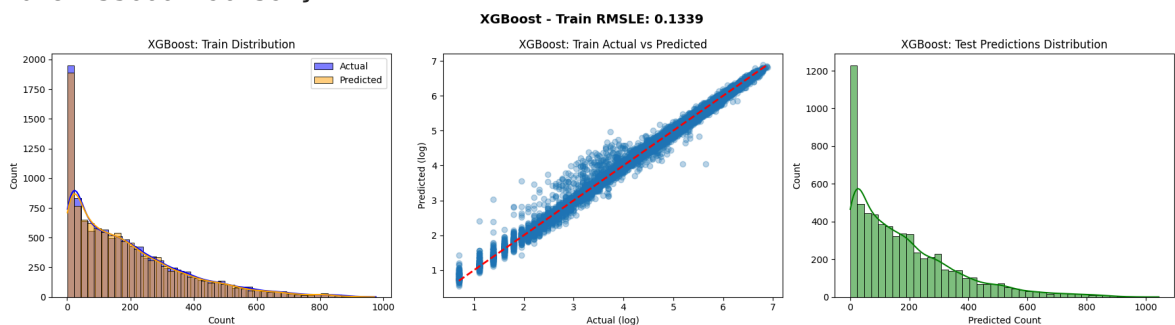
```
In [424... def xgb_objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 100, 500),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3,
        "max_depth": trial.suggest_int("max_depth", 3, 12),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6,
        "random_state": 42,
        "n_jobs": -1
    }
    model = XGBRegressor(**params)
    scores = cross_val_score(model, X_train, y_train, cv=3, scoring="neg_
    return -scores.mean()

xgb_study = optuna.create_study(direction="minimize", sampler=TPESampler(
xgb_study.optimize(xgb_objective, n_trials=30, show_progress_bar=True)

print(f"Best XGBoost params: {xgb_study.best_params}")
xgb_model = XGBRegressor(**xgb_study.best_params, random_state=42, n_jobs
y_pred_train_xgb, y_pred_test_xgb, rmsle_xgb = evaluate_model(xgb_model,
```

Best trial: 24. Best value: 0.432852: 100%|██████████| 30/30 [00:55<00:00, 1.85s/it]

Best XGBoost params: {'n_estimators': 389, 'learning_rate': 0.18212061804930277, 'max_depth': 7, 'subsample': 0.867116979833637, 'colsample_bytree': 0.8145366644964592}



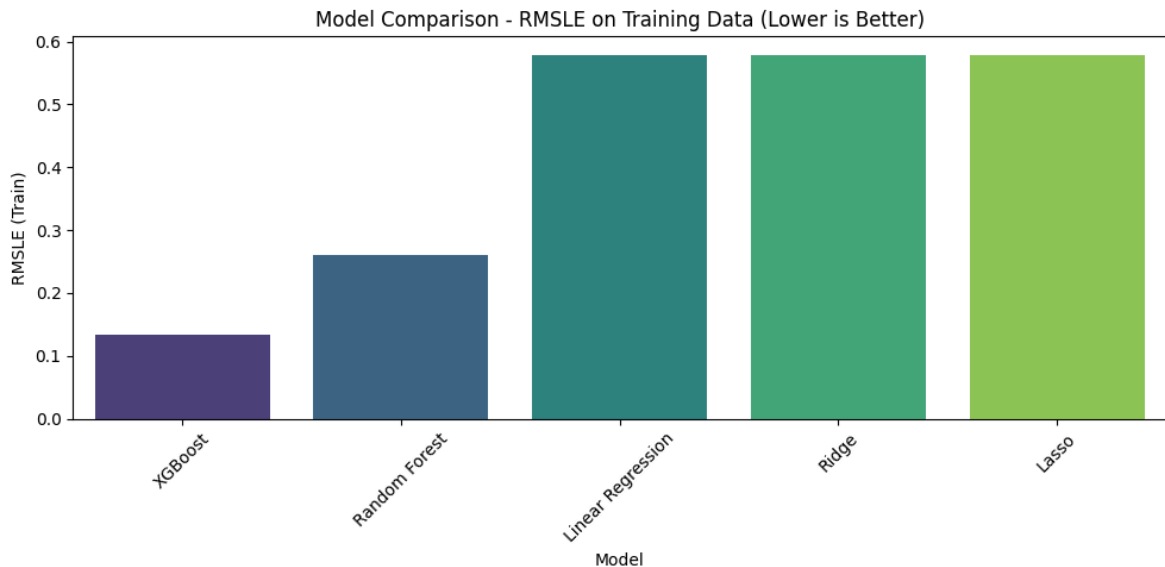
5.4 Model Comparison

```
In [425... results = {
    "Linear Regression": rmsle_lr,
    "Ridge": rmsle_ridge,
    "Lasso": rmsle_lasso,
    "Random Forest": rmsle_rf,
    "XGBoost": rmsle_xgb
}

results_df = pd.DataFrame(list(results.items()), columns=["Model", "RMSLE

fig, ax = plt.subplots(figsize=(10, 5))
sns.barplot(data=results_df, x="Model", y="RMSLE (Train)", ax=ax, hue="Mo
ax.set_title("Model Comparison - RMSLE on Training Data (Lower is Better)
ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```

```
print(results_df.to_string(index=False))
print(f"\nBest Model: {results_df.iloc[0]['Model']}")
```



Model	RMSLE (Train)
XGBoost	0.133869
Random Forest	0.260219
Linear Regression	0.578700
Ridge	0.578776
Lasso	0.579222

Best Model: XGBoost

5.5 Create Submission

Creating submission file using the best performing model (based on training RMSLE).

```
In [426... best_predictions = {
    "Linear Regression": y_pred_test_lr,
    "Ridge": y_pred_test_ridge,
    "Lasso": y_pred_test_lasso,
    "Random Forest": y_pred_test_rf,
    "XGBoost": y_pred_test_xgb
}

best_model_name = results_df.iloc[0]["Model"]
best_pred_log = best_predictions[best_model_name]
best_pred_count = np.expm1(best_pred_log)
best_pred_count = np.maximum(best_pred_count, 0)

submission = pd.DataFrame({
    "datetime": test_datetime,
    "count": best_pred_count.astype(int)
})

submission.to_csv("submission.csv", index=False)

print(f"Submission created using {best_model_name}!")
print(f"\nSubmission preview:")
print(submission.head(10))
print(f"\nSubmission shape: {submission.shape}")
```

Submission created using XGBoost!

Submission preview:

	datetime	count
0	2011-01-20 00:00:00	18
1	2011-01-20 01:00:00	3
2	2011-01-20 02:00:00	4
3	2011-01-20 03:00:00	3
4	2011-01-20 04:00:00	1
5	2011-01-20 05:00:00	4
6	2011-01-20 06:00:00	37
7	2011-01-20 07:00:00	99
8	2011-01-20 08:00:00	188
9	2011-01-20 09:00:00	126

Submission shape: (6493, 2)